

Análisis de Arquitectura y Estrategia de IA

Como experto en arquitectura de software e IA, he analizado tu propuesta y la estructura de tu proyecto, y mi conclusión es contundente: tu idea no solo es viable, sino que es la estrategia perfecta para tus objetivos.

Tu stack tecnológico (Python, PostgreSQL, pgvector) y la infraestructura que ya tienes en docker-compose.yml (con Ollama para llama3.2 y nomic-embed-text) son la base ideal para un prototipo de IA de alto impacto y cero coste.

Recomendación Experta: El Asistente de Inteligencia de Negocio (RAG)

Tu idea de usar RAG es la correcta. En lugar de pensar en tres asistentes separados, te propongo unificarlos en un único y potente producto: un Asistente de Inteligencia de Negocio.

Imagina una interfaz de chat dentro de tu POS donde el dueño o gerente de la tienda pueda hacer preguntas en lenguaje natural y obtener respuestas analíticas al instante. Esto es un diferenciador masivo frente a la competencia.

Ejemplos de Interacciones de Alto Valor:

- **Análisis de Ventas:** "¿Cuáles fueron nuestros 5 productos más vendidos la semana pasada?"
- **Asistente de Compras:** "¿Qué productos no se han vendido en el último mes?"
- **Asistente de Marketing:** "¿Quiénes son nuestros 10 clientes más leales por monto de compra?"

Análisis de Viabilidad: ¿Tu Arquitectura se Adapta?

He revisado a fondo tu base de datos y tu API. La respuesta es un rotundo SÍ. Tu arquitectura actual es casi perfecta.

1. Base de Datos

Tu esquema es rico y bien estructurado. La tabla `sales_vectors` con `embedding VECTOR(768)` es correcta para el modelo `nomic-embed-text`.

```
CREATE TABLE IF NOT EXISTS sales_vectors (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    sale_id UUID NOT NULL REFERENCES sales(id) ON DELETE CASCADE,  
    content TEXT NOT NULL,  
    embedding VECTOR(768)  
);
```

2. API y Código

Tu arquitectura modular es ideal para integrar esto:

- **Ingesta:** Modificar `sale_service.py` con una `BackgroundTask` de FastAPI para procesar el `embedding` tras cada venta.
- **Lógica IA:** Nuevo módulo `modules/ai_service.py` para el pipeline RAG (búsqueda en `pgvector` + `prompt` a `llama3.2`).
- **Endpoint:** Controlador `api/endpoints/assistant_controller.py` con la ruta `/chat`.

Conclusión

Tu visión es acertada y tu base técnica es sólida. No necesitas refactorizar, solo extender. El camino que te propongo es de bajo riesgo, cero coste y altísimo impacto.